

RAG EVALUATION PLATFORM • BUILT ON RAGAS

The Answer That Sounds Right

A story about trusting what your AI tells customers — and catching it when it's wrong.

A Demo Walkthrough

It never says “I don’t know.”

Every AI chatbot answers fast, and answers confidently. That's exactly what makes a wrong answer dangerous — it doesn't look wrong. It reads clean, on-topic, well-formatted. Exactly like a right answer.

“Confidently wrong looks exactly like confidently right.”



“Standard items can be returned within 30 days...”

Reads confident. Is correct.



“Clearance items can be returned within 30 days too...”

Reads just as confident. Is false.

“Someone asks it 30 questions and says it looks right.”



20–30 questions, read by hand

No scale past a handful of test cases before a release.



Same 2–3 tired reviewers

Subjective, inconsistent, and skimmed at the end of a long day.



Never repeated after launch

Quality regressions go unnoticed — until someone complains.

The first person to catch a bad answer is usually the customer.

Every answer, scored before it reaches a customer.

Instead of a person reading an answer and guessing, this platform scores every answer against four industry-standard metrics — automatically, in seconds, from one question to a full regression suite.

Built on RAGAS — a peer-reviewed, open-source standard. Not a black box.

1

Connect

Point at your RAG API, or paste answers manually

2

Configure

Pick a provider, save credentials once

3

Evaluate

Run one question or a full batch — get a scored report

Four metrics. Plain-language questions.

1 **Response Relevancy**

Did it actually answer what was asked?

2 **Faithfulness**

Is it grounded in real source data — or invented?

3 **Context Precision**

Is it retrieving the right material, without noise?

4 **Context Recall**

Is it finding everything it needs to answer fully?

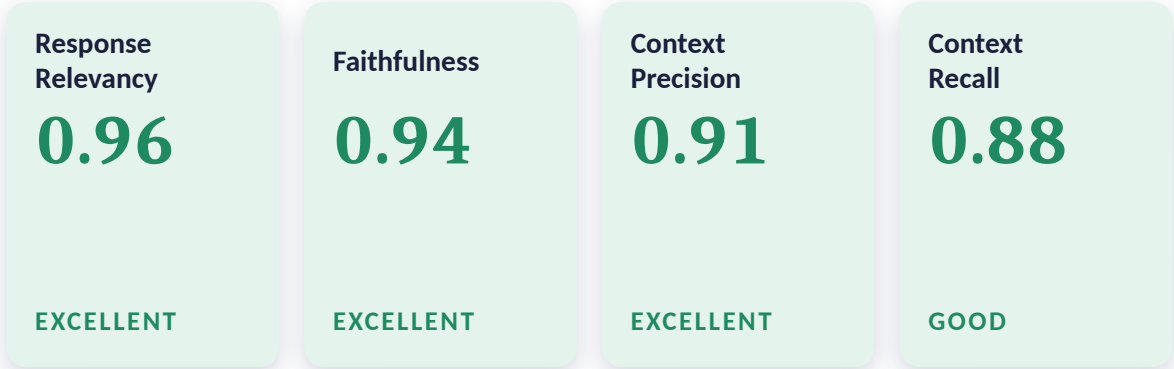
When it's right, you'll know exactly why.

QUERY

“What's your return window for a standard unopened item?”

BOT'S ANSWER

“You can return a standard, unused item within 30 days of delivery for a full refund, as long as it's in its original packaging.”



Green across the board. Good — but not the interesting part yet.

Confidently wrong. And now, caught.

QUERY

"I bought a clearance jacket — can I return it within 30 days like everything else?"

BOT'S ANSWER

"Yes, clearance items can be returned within 30 days for a full refund, just like standard items." — Actually false: clearance is final sale.

Response
Relevancy

0.91

EXCELLENT

Faithfulness

0.32

POOR

Context
Precision

0.88

GOOD

Context
Recall

0.83

GOOD

Relevancy and retrieval are fine. Faithfulness collapses — the answer contradicts its own source.

It tells you where to look next.

Retrieval was healthy — the right paragraph was there. But the generated answer wasn't grounded in it. So the diagnosis narrows to two places.

Healthy

Retrieval

Pulled the right paragraph, no noise.

Check here

Augmentation

How the retrieved text was handed into the prompt.

Check here

Generation

How the model used it once it got there.

A hover tooltip shows the judge model's own reasoning for the score — every time.

One click. Unattended. At scale.

BY HAND

An afternoon

to read 24 cases carefully

WITH THE PLATFORM

One click

runs unattended in the background

BATCH AVERAGE — 24 TEST CASES

Response Relevancy	0.93 EXCELLENT
Faithfulness	0.81 GOOD
Context Precision	0.88 GOOD
Context Recall	0.79 GOOD

SUGGESTED FOCUS ACROSS THIS BATCH

Augmentation + Generation — 3 of 24 cases share the exact fingerprint of the clearance question.

Retrieval — 2 of 24 point at knowledge-base gaps instead.

Evidence, not a screenshot that gets lost.



Shareable

Every case — single and batch — in one report: input, answer, contexts, all four scores, timestamped.



Sign-off ready

Hand it to whoever approves a release. It stays around as a record.



Accumulates

UI runs and CI test-suite runs land in the same place — trend quality across releases, not just once.

On the roadmap: Query Generator

Today, someone still has to hand-write the test questions and correct answers before evaluation can run. Query Generator reads a knowledge base directly and generates that test set automatically — so even that manual step goes away.

TODAY

**Someone writes
test cases by hand**



NEXT

**Platform generates
them from your content**

Find it in twenty seconds. Before launch.

4

metrics scored automatically

1

open standard — RAGAS, not a black box

Minutes

not afternoons, to check a full batch

The question isn't whether this works — it's whether a team wants to find its next “clearance jacket” problem in twenty seconds, before launch, or the way most teams still do: after a customer already has.

Thank You

Questions & discussion